Marks	10.25/15.00
Grade	**6.83** out of 10.00 (**68**%)

Question **1**

Incorrect

Mark 0.00 out of 1.00

Flag question

ADT Map and ADT MultiMap have a pretty similar interface. Which of the following operations do not have the same number of parameters?

Select one or more:

- [x] add
- [ ] remove
- [ ] size
- [ ] iterator
- [ ] isEmpty

Your answer is incorrect.

In case a Map, since keys are unique, when we remove an element, it is enough to specify its key. But in a Multimap a key can have multiple values, so when we remove something, we need to specify both the key and the value.

The correct answer is: remove

Question **2**

Correct

Mark 1.00 out of 1.00

Flag question

ADT Multimap can be represented retaining unique elements and for each of them a dynamic array of associated values.

This is similar to having a simple Map, in which the values are of type dynamic array. This version is actually how you can "simulate" a MultiMap in programming languages where this container does not exist but the Map does.

For which operation would we have different types for the input parameters for these two versions?

Select one or more:

- [x] remove ✔
- [ ] search
- [x] add ✔
- [ ] size

Your answer is correct.

Operation add takes as parameter a key and a value for both containers. But for the MultiMap, value is actually just a value, while for Map, in this setting a value is a dynamic array, so the parameter for add would be a key and a dynamic array.

Operation remove has different parameters already on the interface of Map (just the key) and MultiMap (a key and a value).

Operation search is the same, you pass only a key as parameter.

Operation size takes no parameter, so this is the same as well.

The correct answers are:
add,

remove

Question **3**

Correct

Mark 1.00 out of 1.00

Flag question

Which of the following containers does not have iterator?

Select one or more:

- [ ] SortedBag
- [ ] SortedMap
- [ ] Bag
- [ ] SortedSet
- [x] Priority Queue ✔
- [ ] SortedMultiMap
- [ ] Map
- [ ] MultiMap
- [ ] Set
- [x] Stack ✔
- [x] Queue ✔

Your answer is correct.

ADT Stack, Queue and PriorityQueue have so called restricted access to the elements, meaning that we cannot see their content, so they do not have iterator.

The correct answers are:
Stack,

Queue,

Priority Queue

Question **4**

Correct

Mark 1.00 out of 1.00

Flag question

In a List there are positions and the interface of the List contains operations which take a position as parameter. All these operations require the position to be valid. For which operation is the definition of a *valid position* different from the definition considered by other operations?

Select one or more:

- [x] addToPosition ✔
- [ ] getElement
- [ ] remove
- [ ] setElement

Your answer is correct.

In general a position is valid if it represents an actual element from the list. For a list with 5 elements, valid positions are 1, 2, 3, 4, 5 (assume 1-based indexing). However, when we add a new element, it can be added at the end of the list, at position 6 in the above example, so for operation addToPosition we have a different definition of when is a position valid.

The correct answer is: addToPosition

Question **5**

Correct

Mark 1.00 out of 1.00

Flag question

What is the main difference between ADT IndexedList and ADT IteratedList?

Select one or more:

- [x] The way in which positions are defined ✔
- [ ] IndexedList does not have an iterator, while IteratedList has.
- [ ] There is no difference between them, they denote the same container, but it can be found under different names in different books.
- [ ] In an IteratedList there are no positions, while in IndexedList there are.
- [ ] IndexedList is implemented on a dynamic array, while IteratedList is implemented on a linked list

Your answer is correct.

The most important property of ADT List is the existence of positions (it is the only container which has positions). The difference between IndexedList and IteratedList is how do we define the notion of a position: in case of IndexedList positions are represented by integer number, while in case of IteratedList positions are represented by iterators.

The correct answer is: The way in which positions are defined

Question **6**

Correct

Mark 1.00 out of 1.00

Flag question

Which operation exists for the ADT IndexedList and does not exist for the ADT IteratedList?

Select one or more:

- [x] iterator ✔
- [ ] position
- [ ] remove
- [ ] size
- [ ] addToEnd

Your answer is correct.

ADT IteratedList has operation *first* which returns an iterator set to the first element of the list. Since this is exactly what operation *iterator* would do, there is no use to have operation iterator as well.

The correct answer is: iterator

The correct answer is: Iterator

Question **7**

Partially correct

Mark 0.83 out of 1.00

Flag question

Which of the following operations from the interface of ADT List no longer exist for ADT SortedList?

Select one or more:

- [x] setElement ✔
- [ ] removePosition
- [ ] size
- [x] position ✖
- [ ] getElement
- [ ] isEmpty
- [x] addToEnd ✔
- [ ] search

Your answer is partially correct.

You have selected too many options.
In a SortedList elements are sorted, so operations like addToEnd make no sense (when an element needs to be added, there is one single position where it should be - according to the relation used for sorting - so you cannot force an element to be added to the end, it might mess up the sorting). Similarly, setElement might mess up the sorting of the elements as well.

The correct answers are: addToEnd, setElement

Question **8**

Correct

Mark 1.00 out of 1.00

Flag question

Which is the correct representation of a singly-linked list node?

Select one or more:

- Node:
  info: TElem
  next: ↑ Node
  prev: ↑ Node

- Node:
  info: TElem
  next: ↑ Node

- Node:
  next: ↑Node

- Node:
  value: TElem

Your answer is correct.

In a singly linked list every node contains an information (of type TElem) and the address of the next node (pointer to a Node).

The correct answer is: Node:
info: TElem
next: ↑ Node

Question **9**

Incorrect

Mark 0.00 out of 1.00

Flag question

We have a singly linked list made of 10 nodes, each node contains 4 bytes of information and a pointer which occupies 4 bytes as well. The first node is at the memory address 1048. What is the address of the last node?

Select one or more:

- [ ] We have no way of knowing
- [ ] 1058
- [x] 1128 ✖
- [ ] 1120

Your answer is incorrect.

Linked list nodes contain pointers because we have no way of computing where a node is (like we did for a dynamic array).

The correct answer is: We have no way of knowing

Question **10**

Correct

Mark 1.00 out of 1.00

Flag question

Assuming that head is a pointer to the first node of a singly linked list, which condition(s) checks if the list is made of exactly 3 nodes? (For a node assume the representation discussed at the lecture). Check multiple solutions if you think that all the conditions from that answer are needed together.

Select one or more:

- [x] head ≠ NIL ✔
- [x] [head].next ≠ NIL ✔
- [ ] [head].next = NIL
- [x] [[[head].next].next].next = NIL ✔
- [ ] [[[head].next].next].next ≠ NIL
- [ ] [[head].next].next = NIL
- [ ] head = NIL
- [x] [[head].next].next ≠ NIL ✔

Your answer is correct.

head is the address of the first node.

[head].next is the address of the second node.

[[head].next].next is the address of the third node.

[[[head].next].next].next] is the address of the fourth node.

If we want our list to have exactly 3 elements, the address of the fourth node should be NIL. This checks that we do not have more than 3 nodes. But we also have to check that the 3 nodes exists, so head ≠ NIL, [head].next ≠ NIL, [[head].next].next ≠ NIL.

The correct answers are: head ≠ NIL, [head].next ≠ NIL, [[head].next].next ≠ NIL, [[[head].next].next].next = NIL

Question **11**

Partially correct

Mark 0.75 out of 1.00

Flag question

For a singly linked list, in general, we store only the head of the list. What operation would have a better complexity if we stored the tail as well?

Select one or more:

- [x] removeEnd ✗
- [ ] size
- [ ] search
- [x] addToEnd ✓
- [ ] getElement

Your answer is partially correct.

You have selected too many options.
For size, normally, you have to count every node.

For search you have to check every node, until we either find the element we are looking for, or we get to the end of the list.

For getElement we have to go through the nodes until we get to the required position. Storing the tail we would have access to the last element, but we don't know its position and even if we knew, this would be another best case, it would not change the overall complexity of the function.

For addToEnd having the tail would reduce the complexity to $\Theta(1)$

For removeEnd having the tail is not useful, because removing the last element means setting the "next" of the previous element to NIL. But we do not have access to the previous element, and getting it means going through all the nodes of the list.

The correct answer is: addToEnd

Question **12**

Partially correct

Mark 0.17 out of 1.00

Flag question

Which of the following operations has a better complexity for a dynamic array than for a singly linked list?

Select one or more:

- [ ] return an element from a given position
- [ ] check if it is empty
- [x] search for an element ✖
- [x] remove the last element ✔
- [ ] find the position of a given element

Your answer is partially correct.

You have correctly selected 1.

return an element from a given position and remove the last element have a $\Theta(1)$ complexity in case of a dynamic array and $O(n)$ and $\Theta(n)$ complexity for a singly linked list.

The correct answers are: return an element from a given position, remove the last element

Question **13**

Question **13**

Incorrect

Mark 0.00 out of 1.00

Flag question

Consider the following implementation of the search function for a singly linked list (assume the representation from Lecture 5), which returns the node with elem, or NIL if elem is not in the list.

```
function search (sll, elem) is:
        current ← sll.head
        while [current].info ≠ elem AND current ≠ NIL execute
                current ← [current].next
        end-while
        search ← current
end-function
```

Which of the following statements is/are correct?

Select one or more:

- [ ] The implementation will not work correctly if the element is not in the list, otherwise it works correctly.
- [ ] The implementation will not work correctly if the element appears multiple times in the list, otherwise it works correctly.
- [ ] The implementation will not work correctly if the element is in the list, otherwise it works correctly.
- [ ] The implementation will not work correctly if the list is empty, but for all other cases it will return the correct answer.
- [ ] The implementation will not work correctly if the element we are looking for is the last, but otherwise it works correctly
- [x] The implementation is correct. ✖

Your answer is incorrect.

Whenever we derefence a node (we access its content) we have to be sure that the node is not NIL. We check if current is NIL in the code, but only after dereferencing it. So if current is NIL, it is going to crash before the condition current ≠NIL, when we check [current].info. When is current going to be NIL? When the element is not in the list (and this includes the case when the list is empty).

The correct answer is: The implementation will not work correctly if the element is not in the list, otherwise it works correctly.

Question **14**

Correct

Mark 1.00 out of 1.00

Flag question

Consider the following implementation for a function to compute the number of nodes in a singly linked list

```
function size(sll) is:
        count ← 0
        current ← sll.head
        while [current].next ≠ NIL execute:
                count ← count + 1
                current ← [current].next
        end-while
        size ← count
end-function
```

Choose the statement(s) that is/are true about function *size*.

Select one or more:

- [x] It is not correct, but if we change the condition from the while loop to current ≠ NIL, it will be correct. ✔
- [ ] It is not correct, but if we change the condition from the while loop to [[current].next].next ≠ NIL, it will be correct.
- [ ] It is not correct, but if we initialize count with 1 it will be correct.
- [ ] It is correct.

Your answer is correct.

The code is not correct. Condition [current].next ≠NIL, checks if current is the last node. And if it is, the while loop stops and this node will not be counted. So, in general, size returns a value which is the actual size - 1. Initializing count with 1 would solve the problem for most inputs, except the empty lists. Because in case of an empty list current is NIL and the condition [current].next ≠ NIL will crash. So initializing count to 1 is not the solution (or it is, if we check before whether the list is empty and return 0 in that case).

The correct answer is: It is not correct, but if we change the condition from the while loop to current ≠ NIL, it will be correct.

Question **15**

Partially correct

Mark 0.50 out of 1.00

Flag question

Consider the following implementation that adds a new node immediately after the first node of a singly linked list.

```
subalgorithm addSecond(sll, elem) is:
        if sll.head ≠ NIL then
                newNode ←allocate()
                [newNode].info ← elem
                [sll.head]. next ← newNode
                [newNode].next ← [sll.head].next
        end-if
end-subalgorithm
```

Pick the statements that are true about the addSecond subalgorithm.

Select one or more:

- [x] If we swap the last two instructions (ignoring end-if and end-subalgorithm), it will work correctly. ✔
- [ ] The code never crashes.
- [ ] If the list has one single element, it will work correctly.
- [ ] It is correct.
- [ ] If the list is empty, it will crash.

Your answer is partially correct.

You have correctly selected 1.

Adding a new element after the first node makes no sense if the list is empty, so this is why we check this condition at the beginning and if the list is empty, we do nothing.

The code is not correct, the problem is with the order of the last two instructions: first we have to set the next of newNode, only after that can we set the next of the head.

What we do now, is a loop. In the last statement, when we set [newNode].next to be [sll.head].next, we make it to point to itself, so we make a cycle.

The correct answers are: If we swap the last two instructions (ignoring end-if and end-subalgorithm), it will work correctly., The code never crashes.